

README

Part 17: Agentic RL — 让模型学会自主使用工具、完成长程任务

- > Part 11 的 GRPO 解决的是"单轮答题"（一个问题一个回答一个奖励）。
- > 2026 年真正的战场是 Agentic RL：模型在多轮循环中调用工具、观察结果、规划下一步，
- > 用整条轨迹的结果作为奖励来学习——字节/B 站/滴滴/NIO 的 2026 JD 都把它写进了原文。
- > 本部分手写这条管线的最小闭环（GPU ~15 秒 / 纯 CPU 约半分钟，内置真实掩码消融对照），
- > 再对照工业框架。
- > 锚点仓库：[\[verl\]\(https://github.com/volcengine/verl\)](https://github.com/volcengine/verl)（multi-turn 支持） ·
- > [\[RAGEN/StarPO\]\(https://github.com/RAGEN-AI/RAGEN\)](https://github.com/RAGEN-AI/RAGEN)（多轮 RL 稳定性） ·
- [\[AgentGym-RL\]\(https://github.com/woooodyy/AgentGym-RL\)](https://github.com/woooodyy/AgentGym-RL)

学习目标

完成本部分后，你将能够：

- 手写 多轮工具调用 rollout（观测 mask、上下文重进）并完成 BC 冷启动 → RL 两阶段
- 实现 轨迹级 GRPO（组内优势广播 + 观测 loss-mask）并解释其工程权衡
- 复现 掩码消融：观测泄露让策略学会"复读观测"而非泛化（附真实数字）
- 配置 verl 的 multi-turn 训练与自定义奖励函数
- 识别 Echo Trap 等 Agentic 陷阱并设计奖励防 hacking 防线

理论背景（导览）

核心三问——多轮轨迹怎么采、观测 token 要不要算 loss、稀疏的轨迹级奖励怎么分配到每个 token——的完整推导与实测放在 01 章展开（问题引入 → 轨迹级 GRPO 数学 → 同 seed 掩码消融），本页只给地图：

概念	一句话	详见
多轮轨迹	问题 → 调工具 → 看结果 → 再调 → 答案，整条轨迹一个奖励	[01 章](01_from_single_turn_to_agent.md)
观测 mask	环境给的 token 不进 loss；观测含答案时更要防"复读"泄漏	[01 章消融实测](01_from_single_turn_to_agent.md)
轨迹级 GRPO	组内标准化优势广播到全部 assistant token	[01 章](01_from_single_turn_to_agent.md)
奖励与稳定性	稀疏 vs 塑形、Echo Trap / StarPO-S、框架与评估选型	[02 章](02_rewards_and_frameworks.md)

章节导航

章节	内容	对应脚本
[从单轮 RLVR 到 Agentic RL](01_from_single_turn_to_agent.md)	多轮轨迹/观测 mask/轨迹级奖励/掩码消融/Echo Trap；手写管线	'01'
[奖励设计与工业框架](02_rewards_and_frameworks.md)	稀疏 vs 塑形奖励、课程学习、verl/slime/rLLM 选型、评估（ τ -bench/GAIA）	—（CLI）

前置知识

- 必须掌握：[Part 11 对齐实战](../Part11_alignment_verl/tutorial/README.md)——GRPO 组内优势、KL 惩罚（本部分轨迹级优势 = 它的广播版，"全同组优势全零"性质直接复用）
- 建议掌握：
 - [Part 8 04 章 PPO/GRPO](../Part8_post_training/tutorial/04_ppo_and_grpo.md)——策略梯度与优势估计的推导（看懂轨迹级 GRPO 的数学）
 - [Part 12 微调实战](../Part12_finetune_llamafactory/tutorial/README.md)——chat template（工具调用协议就是它的扩展）
 - 可选：[Part 14 推理部署](../Part14_inference_vllm/tutorial/README.md)——vLLM/SGLang 引擎背景（verl/slime 的 rollout 层建立在它之上，读框架源码前值得了解）

在 LLM 链路中的位置

Part 11（单轮 GRPO）→ 【本部分: 多轮 + 工具 + 环境 = Agentic RL】→ 真实 Agent 产品

为什么是主战场：OpenAI Deep Research/Kimi-Researcher 披露"端到端 RL on hard tasks"；Kimi 平均 23 次工具调用/回答——会写单轮 GRPO ≠ 会训 Agent，差异全在多轮机制。

环境

脚本 01 纯 CPU 可跑（GPU ~15 秒 / 纯 CPU 约半分钟，零新依赖，seed 固定）。
工业框架（verl multi-turn / verl-agent）用 Docker + 0.5B 模型（Part 11 02 章的环境直接复用）。

学习地图

多轮轨迹采集（工具调用→观测重进上下文）← 点：与单轮的本质差异
↓ 轨迹级 GRPO（组内优势广播到全部 assistant token）
BC 冷启动 → RL（R1 同款两阶段）← 线：冷启动覆盖度决定 RL 能否启动
↓
掩码消融（观测泄漏 → 复读 vs 内化）← 线：同 seed 对照实验的真实数字
↓
奖励设计与稳定性（Echo Trap）→ 工业框架 ← 面

课后作业

[Assignment 17](../assignments/assignment_17/)

相关资源

- 综述：The Landscape of Agentic Reinforcement Learning for LLMs (arXiv 2509.02547)
- Search-R1 (2503.09516) · RAGEN/StarPO (2504.20073) · GiGPO (2505.10978) · ToolRL (2504.13958)
- Silver & Sutton 《Welcome to the Era of Experience》 (DeepMind, 2025)
- [verl multi-turn 文档](https://github.com/volcengine/verl) · [verl-agent (GiGPO 官方)](https://github.com/langfengq/verl-agent) · [AgentGym-RL](https://github.com/woooodyy/AgentGym-RL)

[← 上一章：Part 16 图像/视频生成](../Part16_image_video_generation/tutorial/README.md) |
 全课程结业：[返回总览](../README.md)

到这里，Part 1-17 的全部课程已经走完——从字符级 bigram 一路到 Agentic RL。
 结业导览：回[总览 README](../README.md) 检查"如何判断学完一个 Part"清单，
 然后带着你的实操记录（本课每章的真实数字）进入
 [面试指南 §7b 方向深挖](../docs/llm_interview_guide.md)。

01_from_single_turn_to_agent

01 — 从单轮 RLVR 到 Agentic RL

- > Part 8/11 的 GRPO：一个问题 → 一次回答 → 一个奖励（单轮）。Agentic RL 把游戏
- > 改成了：问题 → 调工具 → 看结果 → 再调 → …… → 最终答案 → 一个奖励。
- > 三个新问题随之而来：多轮轨迹怎么采、观测 token 要不要算 loss、稀疏的轨迹级奖励
- > 怎么分配到每个 token。本章手写最小闭环并逐个回答（跑
- > [scripts/01_toy_agent_grpo.py](../scripts/01_toy_agent_grpo.py)，GPU ~15 秒 / CPU 约半分钟，
- > 内置一组真实掩码消融对照实验）。

学习目标

完成本章后，你将能够：

- 手写 多轮工具调用 rollout（含观测 mask 与上下文重进）
- 实现 轨迹级 GRPO（组内优势广播 + 观测 mask）
- 复刻 R1 两阶段的玩具版并解释每阶段的分工
- 说出 Agentic RL 与单轮 RLVR 的至少三个实现差异
- 识别 Echo Trap 等常见陷阱并设计防范策略

前置知识

必须掌握：

- Part 11：GRPO 组内优势（本章轨迹级优势 = 它的广播版）
- Part 12 01 章：chat template（工具调用协议 = 它的扩展）

理论背景

问题引入：为什么需要 Agentic RL？

单轮 RL 虽然强大, 但只能处理单轮问答:

1. 轮次限制: 无法处理多轮对话
2. 工具限制: 无法调用外部工具
3. 环境限制: 无法与环境交互

Agentic RL 通过多轮工具调用来弥补:

单轮 RL: "一问一答"

Agentic RL: "多轮对话, 调用工具, 与环境交互"

> 类比: 单轮 RL 像是只会回答问题的人, Agentic RL 像是会使用工具的人。工具的能力让解决问题更高效。

数学推导: 轨迹级 GRPO

问题设定:

- 轨迹: $\tau = (s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_T, a_T, r_T)$
- 轨迹级奖励: $R(\tau)$

推导过程:

Step 1: 采样轨迹

从当前策略 π 采样 G 条轨迹: $\tau_1, \tau_2, \dots, \tau_G$

Step 2: 计算轨迹级奖励

$R(\tau_i)$ = 最终奖励 (如任务完成度)

Step 3: 计算优势

$A_i = (R(\tau_i) - \text{mean}(R)) / \text{std}(R)$

Step 4: 策略更新

$L = -\sum \log \pi(a|s) * A$

关键洞察:

- 轨迹级 GRPO 是单轮 GRPO 的自然扩展
- 观测 token 不参与 loss 计算 ($\text{mask}=0$)
- 工具调用也需要学习 ("学会调对工具")

代码实现

1. 多轮轨迹的解剖

运行 [scripts/01_toy_agent_grpo.py](../scripts/01_toy_agent_grpo.py) 验证以下代码。

```
[user: 1 2 1] ← 任务 (mask=0)
<tool_call> multiply 1 2 </tool_call> ← assistant 段 (mask=1)
assistant: → 2 ← 观测 (mask=0)    消融组 A 此处替换为 <mask>
<tool_call> add 2 1 </tool_call> ← assistant 段 (mask=1)
```

```
assistant: → 3 ← 观测 (mask=0)
3 <eos> ← assistant 段 (mask=1) : 最终答案
```

形状追踪：多轮轨迹

```
| 多轮轨迹数据流 |
| |
| 输入: user_message (str) |
| ↓ tokenize |
| user_ids: (n_user,) |
| ↓ generate (到 <tool_call> 或 <eos>) |
| assistant_ids: (n_assistant,) |
| ↓ parse_call |
| tool_call: (str) |
| ↓ run_tool |
| tool_result: (str) |
| ↓ tokenize |
| tool_ids: (n_tool,) |
| ↓ 循环直到 <eos> 或 MAX_TURNS |
| |
| 最终轨迹: |
| ids = [user_ids, assistant_ids, tool_ids, assistant_ids, ...] |
| mask = [0, 1, 0, 1, ...] # user=0, assistant=1, tool=0 |
| |
| 奖励分配: |
| R(trajecory) = 0 或 1 |
| A(每个 assistant token) = (R - mean) / std |
```

• 观测 mask=0 的三重理由：① 工具输出是环境生成的，不是模型的"话"；② 模型无法预测外部结果，算 loss 是浪费容量；③ 更糟——模型可能学会"生成自己期望的观测"（幻觉工具结果）。

2. 手写多轮 rollout（脚本核心 ~50 行）

```
python
for turn in range(MAX_TURNS + 1):
    自回归生成（到 <tool_call> 或 <eos>），token
    记入轨迹、mask=1

    gen = generate_until(policy, cur, stop=["</tool_call>", "<eos>"])
    ids += gen; mask += [1] * len(gen)

    if "<tool_call>" in gen:
```

```

call = parse_call(gen) # 解析失败 → 本轨迹终止
obs = run_tool(call) # 环境执行
ids += tokenize(obs); mask += [0] * ... # 观测 mask=0 (不是模型说的)
else:
break # 给出最终答案, episode 结束

```

3. 轨迹级 GRPO：奖励怎么分配

整条轨迹只有末尾一个 0/1 奖励 → 组内标准化后, **优势广播到该轨迹的全部 assistant token** (工具调用也要学——"学会调对工具"本身就是任务的一部分)：

```

A(每个 assistant token) = (r_trajectory - mean(组内 r)) / std
loss = - Σ (log π(token) × A(token)) / #assistant_tokens

```

4. 实测：BC 冷启动 → RL → 掩码消融 (同 seed 两组对照)

脚本对同一 seed 跑两组实验, 唯一变量是 **mask_observations** (观测内容是否泄漏进策略输入), 评测四种条件: 开卷 (工具可用, 与训练同构) vs 闭卷 (工具拿走, 模型直答), train 任务 (6 个训练组合) vs holdout 任务 (训练中从未出现的 2 个组合)。以下输出逐字来自真实运行 (seed=7, RTX GPU)：

> △ 设备说明: CPU 与 CUDA 的浮点差异会让采样轨迹分岔, 你复跑的具体数字可能
> 有波动 (本机纯 CPU 实测, 三列依次为 开卷 holdout / 闭卷 train / 闭卷 holdout :
> A 组 31.2%/39.6%/28.1%, B 组 0%/2.1%/9.4%) ——但"泄漏组在 holdout/闭卷崩塌"
> 的定性结论在两种设备上均稳定复现。

—— 实验组 A: mask=True (观测→<mask>, 标准做法) ——

冷启动后开卷成功率: 99.0%

round 0: 组平均奖励 = 1.000

round 3: 组平均奖励 = 0.990

round 5: 组平均奖励 = 0.948 ← 熵下降带来的波动 (02 章 Echo Trap 伏笔)

训练后: 开卷 train 99.0% | 开卷 holdout 12.5% | 闭卷 train 43.8% | 闭卷 holdout 25.0%

—— 实验组 B: mask=False (观测原样进策略输入——泄漏) ——

冷启动后开卷成功率: 99.0%

round 5: 组平均奖励 = 1.000

训练后: 开卷 train 96.9% | 开卷 holdout 3.1% | 闭卷 train 10.4% | 闭卷 holdout 0.0%

—— 掩码消融对比 (同 seed, 唯一变量 = 观测内容是否泄漏进策略输入) ——

指标 A: mask=True B: 泄漏(mask=False)

开卷 · train 任务 99.0% 96.9%

开卷 · holdout 任务 12.5% 3.1%

闭卷 · train 任务 43.8% 10.4%

闭卷 · holdout 任务 25.0% 0.0%

- 三个必讲的观察 (对应上面真实数字)：

① 没有 BC 就没有 RL（机制层）：随机初始化的策略采不出合法工具调用 → 奖励恒 0 → 组内 std=0 → GRPO 无梯度（稀疏奖励死锁）。R1 论文的 cold start SFT 就是解这个。我们实测过反面：把 BC 示范从 6 个组合砍到 2 个、其余组合从未见过示范——RL 六轮组平均奖励纹丝不动卡在 0.33（未见组合全组失败 → 零方差 → 零梯度）；再把 BC 步数调软（40 步，策略更“软”便于探索），RL 才把 0.33 推到 0.50。**冷启动的覆盖度和策略熵，直接决定 RL 能不能启动**。

② 示范与 rollout 同构时，BC 不只教格式还教会任务：本脚本示范轨迹与 rollout 完全同构，120 步 BC 直接把开卷 train 拉到 99.0%——6 个组合“背下来”即可。此时组内几乎全对 → std≈0 → GRPO 梯度≈0，RL 在训练组合上边际增益小是正常现象（不是 bug；真实场景里任务分布大得多，BC 无法覆盖，RL 才有空间）。

③ 消融的信号不在 train，在 holdout 与闭卷：泄漏组 B 在训练组合上不差（96.9%），但 holdout 3.1%、闭卷 10.4%/0.0%。机理：第二次观测本身就等于最终答案，泄漏组的最优解是“复读前一个观测数字”——它的答案 token 从未被训练成 (a,b,c)→答案 的函数；组合一没见过（holdout）或观测一拿走（闭卷）就现形。mask 组 A 观测不可见，BC/RL 只能靠 user token 把 a*b+c 内化进参数，闭卷 train 仍保住 43.8%。开卷成绩好看 ≠ 学会了计算；泄漏买来的是依赖，不是能力。

- > 玩具判分漏洞（诚实声明）：本脚本奖励取“真实轨迹里最后出现的数字”。
- > 由于第二次工具观测本身就等于最终答案，模型甚至不需要自己给出最终答案——两次
- > 工具调用参数正确即可拿分（观测“冒充”了答案）。真实 RLVR 用三种手段避免：
- > ① 格式约束（`\boxed{}` / `####` 锚定答案必须出现在 assistant 段）；② 工具协议
- > （observation 只进上下文，判分只看 assistant 输出）；③ 答案位置锚定（只解析
- > 结束符前的答案段）。想亲身体会：把判分改成“只看 `<eos>` 前模型自己生成的数字”，
- > 泄漏组的开卷成绩会立刻跌下来。
- > ⚠ 两种 mask，别混淆：本玩具把「观测内容不进策略输入」与「观测 token
- > 不进 loss」合并成一个开关（都是 `mask_observations`）。工业实现（verl/slime
- > 的 multi-turn）是观测进上下文 + loss mask——模型必须读到工具结果才能用工具，
- > 但观测 token 的 loss-mask=0，策略梯度不流过它们。两件事共享同一条原则：
- > 环境给的 token 不该承载策略梯度；本玩具额外演示的是它的孪生问题——
- > 观测里的答案泄漏会让策略学会走捷径。

工程实践

调试展示：常见错误与修复

错误 1：观测 token 误算进 loss / 观测内容泄漏进策略输入

症状：

闭卷（或换环境/换工具）时成功率崩塌；或模型学会“预测/复读工具输出”

原因：两种 mask 语义搞混——观测 token 的 loss-mask 设置 0（策略梯度流过环境 token），或观测里带答案却没有意识到（策略学会走捷径，见上文消融 B 组）

解法：

python

① loss-mask：观测段恒为 0（策略梯度只流向 assistant token）

```
mask = [0] n_user + [1] n_assistant + [0] n_obs + [1] n_assistant
```

② 审计观测内容：观测是否直接包含答案/奖励相关信号？

是 → 要么遮蔽（本玩具 <mask> 替换），要么改判分协议（答案位置锚定）

错误 2：解析失败导致 RL 死锁

症状：

奖励恒 0，训练不收敛

原因：工具调用解析失败，奖励恒为 0

解法：

```
python
```

检查解析是否成功

```
if not parse_success:
```

```
reward = 0.0
```

终止本轨迹，继续下一条

错误 3：Echo Trap（策略熵坍缩）

症状：

奖励曲线看不出来，但 rollout 熵很低

原因：策略熵坍缩到重复模板

解法：

```
python
```

监控 rollout 的熵

```
entropy = -sum(p * log(p) for p in policy_distribution)
```

如果熵太低，使用 StarPO-S

- critic + clip-higher + rollout 过滤

性能数据（实测，环境：RTX GPU / seed=7 / 全程 ~15 秒；纯 CPU 复跑 ~20-60 秒）

策略	开卷 · train	开卷 · holdout	闭卷 · train	闭卷 · holdout
随机初始化（无 BC）	10.4%	—	—	—
仅 BC 冷启动（A: mask）	100.0%	40.6%	36.5%	31.2%
仅 BC 冷启动（B: 泄漏）	100.0%	0.0%	7.3%	3.1%
BC + RL（A: mask）	99.0%	12.5%	43.8%	25.0%
BC + RL（B: 泄漏）	96.9%	3.1%	10.4%	0.0%

- 复现：[python 01_toy_agent_grpo.py](#)（表内后两行 = 脚本直接输出；"仅 BC"行 = 把顶部 **RL_ROUNDS** 改为 0 再跑；"随机初始化"行 = 用未训练的 **TinyPolicy()** 直接调 **open_book_success** 评测）。
- 同一模型、两次评测的采样波动：表中"仅 BC"行开卷 train=100.0% 与上文"冷启动后开卷成功率: 99.0%"是同一个 BC 模型的两次独立评测——评测是对每个任务做 n 次随机 rollout 再算命命中率，两个数字都是真实观测，差异来自评估采样的随机性（不是两次训练）。
- 泄漏组的崩塌在 BC 阶段就已注定：示范轨迹里观测含答案 → "复读观测"从第一步示范起就是最优解；RL 既不造成它、也不修复它。另注意 A 组 RL 后 holdout 从 40.6% 回落到 12.5%——RL 的熵收缩有时会牺牲泛化（02 章稳定性的伏笔）。

常见陷阱

陷阱 1：没有 BC 就没有 RL

症状：训练不收敛，奖励恒为 0

原因：随机策略采不出合法工具调用

解法：先用 BC 冷启动，学会格式再 RL

陷阱 2：观测 mask 错误（loss 泄漏或内容泄漏）

症状：开卷成绩正常，但闭卷/换环境/换任务划分时崩塌（本课消融 B 组：闭卷 train 10.4% vs A 组 43.8%）

原因：观测 token 的 loss-mask 设置 0，或观测内容直接包含答案（策略学会"复读观测"这个捷径）

解法：loss-mask 置 0 + 审计观测内容是否泄漏答案（必要时遮蔽或锚定判分位置）

陷阱 3：Echo Trap

症状：策略熵坍缩到重复模板

原因：多轮 RL 特有的稳定性陷阱

解法：使用 StarPO-S（critic + clip-higher + rollout 过滤）

最佳实践

配置推荐

参数	推荐值	说明
MAX_TURNS	5-10	最大工具调用次数
BC epochs	100-200	冷启动训练轮次
RL rounds	100-200	RL 训练轮次
组大小 G	4-8	每个 prompt 的轨迹数

调试流程

1. 先 BC 冷启动：学会格式
2. 检查解析：确保工具调用能正确解析
3. 监控熵：确保策略不坍塌
4. 逐步增加难度：从简单任务到复杂任务

学完本部分你能...

- 手写多轮工具调用 rollout（含观测 mask 与上下文重进）
- 实现轨迹级 GRPO（组内优势广播 + 观测 mask）
- 复刻 R1 两阶段的玩具版并解释每阶段的分工
- 说出 Agentic RL 与单轮 RLVR 的至少三个实现差异
- 识别 Echo Trap 等常见陷阱并设计防范策略

概念检验

<details>

<summary>Q1: 如果观测 token 误算进 loss，最坏会发生什么？</summary>

A: 模型学会"预测工具输出"——两个失败模式：① 工具输出部分不可预测（外部状态），预测它是浪费容量；② 模型可能生成"自己期望的观测"而非真实调用工具（幻觉工具结果）。这正是 Search-R1 做 retrieved-token masking 的原因。

</details>

<details>

<summary>Q2: 为什么长程任务需要异步 rollout？</summary>

A: 轨迹长度重尾分布——有的 2 步结束、有的 100+ 步（Kimi-Researcher 平均 23 次工具调用）。同步 rollout 里 GPU 要等最长的轨迹，短轨迹早已算完在空转；异步派发（verl rollout.mode=async / AReaL 全异步）让短轨迹先返回继续采样。

</details>

<details>

<summary>Q3: 什么是 Echo Trap（RAGEN 论文）？怎么发现和缓解？</summary>

A: 多轮 RL 特有的稳定性陷阱：策略熵坍塌到重复模板（同样的工具调用循环往复），奖励曲线却看不出来（因为模板可能还拿低奖励）。发现：监控 rollout 的熵与轨迹多样性。缓解：StarPO-S（critic + clip-higher + rollout 过滤）。

</details>

动手实践

<details>

<summary>练习 1: 实现多轮 rollout</summary>

任务：实现一个函数，生成多轮工具调用轨迹。

验收标准：

- [] 输入：策略、用户消息、最大轮次
- [] 输出：轨迹 (ids, mask, reward)
- [] 正确处理观测 mask=0

步骤提示：

```
`python
def multi_turn_rollout(policy, user_message, max_turns=5):
    """
```

Steps:

1. tokenize 用户消息
 2. 循环 max_turns 次
 3. 生成 assistant 回复（到 <tool_call> 或 <eos>）
 4. 如果是 <tool_call>，解析并执行工具
 5. 如果是 <eos>，结束轨迹
 6. 返回轨迹 (ids, mask, reward)
- ```
"""
```

## TODO: Implement

```
pass
`
```

</details>

<details>

<summary>练习 2: 实现轨迹级 GRPO</summary>

任务：实现一个函数，计算轨迹级 GRPO 优势。

验收标准：

- [] 输入：轨迹列表，每条轨迹有 reward
- [] 输出：每条轨迹的 advantage
- [] 正确处理组内标准化

步骤提示：

```
`python
def trajectory_grpo(trajectories):
 """
```

Steps:

1. 提取每条轨迹的 reward
  2. 计算组内均值和标准差
  3. 计算每条轨迹的 advantage
  4. 返回 advantage
- ```
"""
```

TODO: Implement

```
pass
```

</details>

<details>

<summary>练习 3: 实现 BC 冷启动</summary>

任务：实现一个函数，用 BC 冷启动训练策略。

验收标准：

- [] 输入：策略、示范数据
- [] 输出：训练后的策略
- [] 学会格式但不一定学会任务

步骤提示：

```
python
def bc_cold_start(policy, demonstrations, epochs=100):
    """
```

Steps:

1. 遍历示范数据
 2. 计算 BC loss（交叉熵）
 3. 更新策略
 4. 返回训练后的策略
- ```
"""
```

TODO: Implement

```
pass
```

</details>

## 课后作业

完成本章后，去 Assignment 17 完成练习：

[Assignment 17](../../assignments/assignment\_17/)

## 下一步

奖励设计（稀疏 vs 塑形 vs 课程）、稳定性（Echo Trap 深挖）、工业框架选型（verl multi-turn / verl-agent / slime / rLLM）与评估（ $\tau$ -bench/GAIA）。

[02 — 奖励设计与工业框架](02\_rewards\_and\_frameworks.md)

## 02\_rewards\_and\_frameworks

## 02 — 奖励设计与工业框架（Agentic RL 的工程全景）

- > 01 章跑通了最小闭环。本章补齐工程决策：长程任务的奖励怎么设计、
- > 训练不稳定怎么治（Echo Trap）、框架怎么选、怎么评估。

# 学习目标

完成本章后，你将能够：

- 选型奖励类型（稀疏结果 / ToolRL 式塑形 / ORM / 课程学习）并说明各自的适用场景与代价
- 设计 Agentic 奖励的防 hacking 防线（调用去重 / 轮数上限 / 环境沙盒 / 轨迹抽样审查）
- 识别 Echo Trap 的症状并说出 StarPO-S 的三个缓解件
- 配置 verl 的自定义奖励函数（`compute_score`，操作型练习）
- 给出框架与评估基准的选型决策（verl 起步 → slime/verl-agent 进阶； $\tau$ -bench / GAIA L1 起步）

## 前置知识

- 01 章：多轮轨迹、观测 mask、轨迹级 GRPO（[传送门](01\_from\_single\_turn\_to\_agent.md)）
- Part 11：GRPO 组内优势与"全同组优势全零"性质（本章 StarPO-S 的 rollout 过滤直接建立在其上）

### 1. 奖励设计谱系（长程任务的核心难题）

| 类型           | 做法                        | 优          | 劣                              |
|--------------|---------------------------|------------|--------------------------------|
| 稀疏结果奖励       | 只看最终答案对错（Search-R1）       | 简单、不可作弊    | 10+ 轮轨迹的 credit assignment 噪声大 |
| 过程/塑形奖励      | ToolRL：格式奖励 + 每步正确性，细粒度分解 | 信号密、收敛快    | 设计不当会引入偏差                      |
| ORM（结果监督 RM） | 训练奖励模型打分                  | 可泛化到难形式化任务 | RM 可被 hack、有偏差                 |
| 课程学习         | WebRL：从失败任务再生任务，逐步加长      | 与模型能力同步演进  | 系统复杂                           |

- ToolRL 的实证（2504.13958）：把奖励分解为"格式正确 + 参数正确 + 结果正确"的细粒度塑形，显著优于单一结果奖励。

### 常见陷阱：reward hacking（Agentic 特有，与 Part 8 07 章污染是近亲）

症状：复读机式工具调用循环、调用"回显 prompt"的工具、打印预期答案作弊、钻环境 bug——分数上去了，任务没完成。

原因：奖励只度量了任务的"代理指标"，而多轮 + 工具的环境里可钻的空子远多于单轮——策略学到的是捷径，不是能力（分数与能力脱钩，故与污染是近亲）。

解法：防线四件套——调用去重、轮数上限+溢出惩罚、环境沙盒、轨迹抽样人工审查。

### 调试展示：常见错误与修复

错误 1：reward 全零组占比过高，训练停滞

症状：

组平均奖励长期贴 0，策略几乎不更新（01 章"没有 BC 就没有 RL"的死锁）

原因：合法轨迹太少 → 大量组内奖励全 0 → 组内 std=0 → GRPO 优势全零，这些组对 loss 无贡献（01 章实测过的反面：BC 覆盖不足时，RL 六轮组平均奖励纹丝不动）

解法：

python

训练中统计全零组占比——占比过高说明"探索不出合法轨迹"，而不是"策略差"

`zero_frac = sum(1 for rs in group_rewards if sum(rs) == 0) / len(group_rewards)`

先 BC 冷启动教会格式；仍偏高则扩大示范覆盖 / 提高组大小 / 降低任务难度

错误 2：工具调用解析正则漏配

症状：

轨迹大量提前终止、奖励恒 0——但人工看模型输出"格式明明是对的"

原因：解析正则与协议不匹配（协议是空格分隔、正则却按 JSON 逗号写；或 BC 示范本身不合法 → parse 永远失败——01 章 `parse_call` 注释点名的坑）

解法：

python

开训前用一批示范轨迹过 parser，先统计解析成功率（协议一处改动要全链路同步）

`ok = sum(parse_call(t) is not None for t in demo_texts) / len(demo_texts)`  
`assert ok == 1.0, "示范都解析不了，RL 一定死锁"`

错误 3：rollout 长度截断丢最终答案

症状：

长轨迹批量 0 奖励，且与"模型答错"无关——被截断的轨迹根本没有答案段

原因：上下文/步数上限先于 <eos> 触发（01 章脚本的 rollout 有 60 token 上限、闭卷探针 24 步上限），截断轨迹按"无答案 = 0 分"判分——模型被冤枉

解法：

python

区分"答错"与"没答完"：截断单独计数，必要时给部分分或不计入分母

truncated = (len(gen) >= MAX\_STEPS) and (EOS not in gen)

监控截断率；超限走截断/摘要或 partial rollout（见下方 Q2 的三种工程处理）

2. 稳定性：Echo Trap 与 StarPO-S

多轮 RL 特有的失败模式（RAGEN 论文 2504.20073）：策略熵坍缩到重复模板（同样的工具调用循环往复）——奖励曲线看不出来（模板可能还拿低分），但探索已死。

发现：监控 rollout 熵 + 轨迹多样性（不同轨迹比例）  
缓解：StarPO-S = critic 辅助 + clip-higher（提高上界探索）+ rollout 过滤（丢弃零方差组）

我们脚本 01 的组内优势在全组时归零（Part 11 的性质），本质是同一现象的单轮版——"无区分度的组没有梯度"。

3. 工业框架选型（2026-08）

| 框架             | star        | 特点                                      | 适合              |
|----------------|-------------|-----------------------------------------|-----------------|
| **verl**       | 23.2k       | multi-turn/tool-agent 支持最全（docs 好）      | 首选入门与生产         |
| **verl-agent** | 2.3k        | GiGPO 官方实现，ALFWorld/TextWorld 玩境        | 小模型 agent RL 研究 |
| slime          | 8.3k        | 智谱系，Megatron+SGLang, custom generate 灵活 | 大规模生产           |
| rLLM           | 5.8k        | harness/sandbox 无关的干净 env API           | 研究原型            |
| AgentGym-RL    | 855         | 多环境开箱                                   | 教学对比            |
| SkyRL / AReaL  | 2.2k / 5.7k | 全异步（长尾轨迹场景）                             | 大规模             |

> 24GB 实操：verl multi-turn + Qwen2.5-0.5B + 计算器/检索工具（Part 11 环境复用）；  
> verl-agent 的 TextWorld 玩境（0.5B/1.5B 友好）。SkyRL/AReaL/AgentGym-RL 按  
> 文档定位 ≥8 卡，引用不实操。

4. 评估（Agentic 版）

| 基准                   | 测什么                    | 24GB 可评        |
|----------------------|------------------------|----------------|
| τ -bench             | agent+模拟用户+策略合规（零售/航空） | 轻量（需 LLM 演用户）  |
| GAIA L1 子集           | 真实问题（推理+浏览+工具）         | 文本子集           |
| AgentBench           | 8 环境（DB/OS/Web...）     | △ 环境重          |
| WebArena / SWE-bench | 自托管网站 / 真实 issue 修复    | 小模型≈0%，大模型+重环境 |

## 学完本部分你能...

- 按任务特征选奖励类型并设计防 hacking 防线
- 识别 Echo Trap 并说出 StarPO-S 的三个缓解件
- 给出框架选型决策 (verl 起步 → verl-agent/slime 进阶)
- 为 agent 模型选评估基准 ( $\tau$ -bench/GAIA L1 起步)

### 概念检验

<details>

<summary>Q1: GiGPO 相比轨迹级 GRPO 解决什么? </summary>

A: 轨迹级优势对长轨迹粒度太粗 (哪个工具调用是关键? 不可知)。GiGPO 在"锚定状态" (跨 episode 出现的相同状态) 上建 step 级分组, 给细粒度 credit 且保持 critic-free (Part 11 04 章 GRPO 思想的 step 级推广)。

</details>

<details>

<summary>Q2: 工具输出动辄几 KB, 多轮后撑爆上下文。三种工程处理? </summary>

A: ① 截断/摘要 (Search-R1 只保留相关片段); ② 观测入上下文但 mask 出 loss 且 history 压缩; ③ partial rollout (上下文满时强制截断轨迹并保留已完成部分的优势)。verl 的 partial rollout 与 slime 的 context engineering 都在此列。

</details>

<details>

<summary>Q3: 零方差组 (组内奖励全同) 为什么没有梯度? 工程上怎么处理? </summary>

A: GRPO 优势 =  $(r - \text{mean}) / \text{std}$ , 组内全同时分子为 0 (且 std 也为 0, 数值上再被 eps 兜底) → 整组优势全零 → 对 loss 无贡献。这不是 bug 而是性质: "无区分度的组没有信息量"。工程处理: ① StarPO-S 的 rollout 过滤 (丢弃零方差组, 不浪费更新); ② 提高组内多样性 (更高采样温度 / 更长上下文); ③ 改用更细粒度的奖励分解 (ToolRL 式格式/参数/结果分) 让组内出现区分度。01 章脚本里 BC 饱和后 **round** 组平均奖励  $\approx 1.0$ 、std  $\approx 0$  的现象就是它的实例。

</details>

### 动手实践

<details>

<summary>练习 1: 给 verl 写一个自定义奖励函数 (操作型) </summary>

任务: 在 verl 里用自定义 **compute\_score** 替换默认奖励 (Part 11 02 章的 Docker 环境可直接复用), 实现"ToolRL 式三分奖励": 格式分 + 答案正确分。

验收标准:

- [] 函数签名 **compute\_score(data\_source, solution\_str, ground\_truth, extra\_info=None) -> float**
- [] 对 **The answer is 42**、**\boxed{42}**、**#### 42** 三种格式都能抽到答案
- [] 非法/无答案输出返回 0.0 而不是抛异常 (奖励函数崩溃会拖垮整个训练)
- [] 在 verl 配置里通过 **custom\_reward\_function.path** 指向你的文件并跑通一个 sanity batch (日志里能看到非零奖励)

步骤提示:

```
`python
import re
```



```
def compute_score(data_source, solution_str, ground_truth, extra_info=None):
```

Step 1: 依次尝试 `\boxed{...}` / `#### ...` / `"answer is ..."` 三种抽取

Step 2: 抽到则与 `ground_truth` 数值比较 (float 化, 容忍千分位逗号)

Step 3: ToolRL 式分解: 格式分(抽到答案) 0.2 + 正确分 0.8

Step 4: 任何异常路径都 return 0.0

...

</details>

<details>

<summary>练习 2: 在本课玩具上实现 ToolRL 式奖励分解 (本地可跑) </summary>

任务: 把 `[scripts/01](../scripts/01_toy_agent_grpo.py)` 的 0/1 结果奖励分解为 "格式分 + 参数分 + 结果分", 对照原始稀疏奖励跑 6 轮, 比较组平均奖励曲线的前 2 轮。

验收标准:

- [] `decomposed_reward(real_ids, task) -> (fmt, args, result)` 三元组, 各分量  $\in [0,1]$
- [] 格式分 = 合法工具调用数 / 2; 参数分 = 调用参数与 (a,b)/(p,c) 匹配的比例
- [] 总奖励 =  $0.2 \times \text{fmt} + 0.3 \times \text{args} + 0.5 \times \text{result}$  (权重可调, 说明你的理由)
- [] 打印两种奖励的 round 0-1 组平均奖励对照: 分解版应明显更高 (信号更密), 最终成功率不低于稀疏版

步骤提示:

Step 1: 在 rollout 的判分处, 把 `re.findall(r"\d+", ...)` 的单一判分替换为逐段判分 (real\_ids 里 call 段与 obs 段的边界在构建时可记录)

Step 2: 格式分: `parse_call` 成功次数 / `MAX_TURNS`

Step 3: 参数分: `multiply` 的 `args == (a,b)`? `add` 的 `args == (a*b, c)`?

Step 4: 结果分: 最后一个数字 == `answer` (保留原判分)

Step 5: 两种奖励各跑一遍 `run_experiment`, 对比 `curve[:2]` 与最终成功率

...

</details>

## 扩展思考

没有标准答案——每个问题都值得动手验证后再下结论。

思考 1: GiGPO 的"锚定状态"搬到本课玩具会是什么样?

01 章轨迹里, `multiply` 调用后的 `[user | call1 | obs1]` 前缀是天然的锚定状态——

所有任务在"决定第二个调用"时面对的是同构局面。你会怎么在这个状态上建 step 级分组? 轨迹级 GRPO 与 step 级分组各自适合什么轨迹长度/任务结构?

(提示: 锚定状态要求跨 episode 可匹配——本玩具靠固定协议长度, 真实环境靠什么?)

思考 2: 环境设计反过来决定消融结果。

01 章的判分漏洞 (第二次观测=答案, 可冒充最终答案) 让泄漏组开卷也能拿分。如果把

判分改成"答案位置锚定"（只看 `<eos>` 前模型自己生成的数字），消融表会怎么变？  
泄漏组的开卷 · train 还能到 96.9% 吗？这说明了"环境/判分协议"与"算法"之间是什么关系？（动手：改两行判分代码即可验证你的预测。）

思考 3：多轮 credit assignment 的下一站。

10+ 轮轨迹里，第一个工具调用与最终奖励之间隔着几千 token，轨迹级优势把它们一视同仁地强化/削弱。除了 GiGPO，你还能想到哪些思路（过程奖励模型 / 轮次折扣 / 树搜索重排 / 让模型自己生成子目标）？每种思路各引入什么新问题（奖励模型可被 hack？折扣系数难调？搜索开销爆炸？）。

## 课后作业

[Assignment 17](../..../assignments/assignment\_17/)

## Part 17 完结：Agentic RL 是 2026 后训练 JD 的第一关键词——你现在拥有

从机制手写（脚本 01）到框架实操（02 章路线）的完整入门。继续：  
[面试指南 § 7b 方向深挖](../..../docs/llm\_interview\_guide.md)。

[← 上一章](01\_from\_single\_turn\_to\_agent.md) | [Part 17 README](README.md)